Calepin is now a static website generator

Two weeks ago, I introduced *Calepin*, a tool that could turn a standard Typst file into a computational notebook, in the spirit of Quarto or Jupyter. Today, I am pleased to introduce the second **major** feature of *Calepin*: static website generation.

This means that *Calepin* can now take a directory of ordinary `.typ` files and turn it into a complete website: HTML pages, optional PDF versions, navigation, feeds, search, assets, and all the small files that static hosts expect.

What is static website generation?

A static website generator builds a site ahead of time. Instead of running a web application on a server for every visitor, it converts source files into plain HTML, CSS, JavaScript, images, and other assets. The result can be served from almost anywhere: GitHub Pages, Netlify, Cloudflare Pages, an S3 bucket, or a simple web server.

Static sites are fast, easy to host, and pleasantly robust. There is no database to maintain, no application server to patch, and no runtime dependency between your writing and your readers. You write the source files, run a build command, and publish the generated directory.

With *Calepin*, the source files are Typst files. This is the important part. The same language you use for a paper, report, lecture note, slide deck, or computational notebook can now be used for a project website or documentation site.

What can it do?

A *Calepin* website starts with a directory and a `calepin.toml` file:

```
calepin new website my_site/
calepin compile my_site/
```

From there, you can write pages in Typst and let *Calepin* handle the website machinery. Pages can share a theme, appear in menus and sidebars, expose metadata for listings, and be rendered to HTML, PDF, or both.

This website was built that way. The documentation pages, the landing page, the navigation, the theme, and the feeds all come from Typst sources.

Best features

- **Pure Typst authoring.** There is no Markdown layer and no Pandoc translation step. Your pages are `.typ` files, so you can use Typst's layout, styling, math, figures, bibliographies, and scripting directly.
- **One tool for notebooks and websites.** A page can be a document, a notebook, or both. You can execute code chunks, capture figures and tables, and publish the results as part of a website.
- **HTML with optional PDF twins.** Website pages render to HTML by default, and any page can also produce a matching PDF. This is useful for documentation, course notes, handouts, reports, and anything readers may want to print or cite.
- **Themes and layouts.** *Calepin* ships with built-in themes, and themes can be customized or ejected into your project. This keeps the default path simple without hiding the HTML and CSS from people who want control.

- **Navigation from configuration or pages.** Menus and sidebars can be declared in `calepin.toml`, while page metadata and `calepin.pages()` make it easy to build indexes, publication lists, blog archives, course schedules, and custom navigation.
- **Blog-friendly metadata and feeds.** Pages can carry dates, tags, descriptions, authors, drafts, and any other metadata you need. *Calepin* can generate Atom and RSS feeds for sites that publish updates.
- **Multilingual websites.** Pages can be grouped across languages, and themes can expose translation links so readers can move between versions of the same page.
- **Static assets and host files.** Images, downloads, favicons, `robots.txt`, `sitemap.xml`, and `404.html` fit into the same build. The output is ready for common static hosting services.
- **Fast local development.** `calepin watch` rebuilds changed pages incrementally, and `calepin serve` previews the site locally. Use both together while writing:

```
calepin watch my_site/ my_site/ --serve --open
```

- **Search and minification.** For larger sites, *Calepin* can build a Pagefind search index and minify the generated HTML, CSS, and JavaScript.

Why this matters

I like Typst because it is a coherent writing system. It is pleasant for prose, serious enough for technical documents, and programmable when the document needs structure. The notebook feature made Typst useful for computation-heavy documents. Static website generation extends the same idea to publishing.

A *Calepin* project can now be a research note, a report, a course website, a software manual, a blog, a slide deck, and a computational notebook, all with the same source language and the same command-line tool.

That is the goal: fewer formats, fewer conversions, and more time spent writing.